People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

Mohamed Seddik Ben Yahia University - Jijel

Faculty of Exact Sciences and Computer Science

Department of Computer Science

Bachelor's Degree Project

Design and Implementation of a Web-Based School Management System

School Management System

Level: 3rd Year Bachelor's Degree

Specialty: Computer Science

Academic Year: 2025 / 2026

Prepared by (G16):

Bouremouz Mouhammed

Debieche Amine

Bessibes Mouin

Chaabani Sidahmed

Bouzekria Sohib

Meriche Chemseddine

Supervised by

Dr. El Hillali Kerkouche

Executive Summary

This report presents the design and implementation of a web-based School Management System. The proposed platform centralizes administrative, academic, financial, and communication data in one structured system. Each user role accesses a dedicated workspace that matches its responsibilities.

The system follows a layered RESTful architecture: an HTML5, CSS3, and vanilla JavaScript frontend; a Python FastAPI backend exposing modular REST endpoints; and a MySQL relational database for persistence. The application supports six main profiles: administrator, receptionist, accountant, teacher, student, and parent.

Keywords: school management, web application, FastAPI, MySQL, REST API, UML, role-based architecture.

Area	Summary
Objective	Automate daily school management processes and provide each role with an appropriate workspace.
Scope	Users, students, parents, teachers, classes, enrollments, attendance, grades, resources, fees, payments, messages, notifications, and posts.
Method	Existing-system study, requirements analysis, UML design, frontend/backend implementation, and scenario-based validation.
Result	A modular web application with role-based dashboards, a REST API, and a relational database.

System Overview and Development Methodology

This thesis-style report documents the complete software engineering process used to design and implement the School Management System. It begins with the study of the school management domain and the limitations of manual or disconnected tools, then formalizes the functional and non-functional requirements.

The design chapter presents the system architecture, database model, UML diagrams, API organization, security considerations, and user-interface structure. The implementation chapter describes the development environment, source-code organization, application screens, and validation scenarios. The final result is a maintainable, extensible, role-based school management platform.

Table of Contents

List of Figures and Diagrams

General Introduction

General Context

Digital transformation has become a major requirement for educational institutions. Schools manage a large amount of sensitive data every day, including student files, enrollments, schedules, attendance, grades, resources, payments, and communication between stakeholders. When this information is spread across paper registers, spreadsheets, and informal messages, follow-up becomes slower and less reliable.

A centralized web application is an appropriate response to this problem. It provides a shared information base, automates repetitive tasks, traces important operations, and gives each user profile a dedicated interface.

Problem Statement

Traditional school management suffers from data dispersion, slow access to information, fragmented academic follow-up, manual financial errors, and unstructured communication between administrators, teachers, students, parents, and financial staff. The central question is therefore: how can we design a reliable, clear, and extensible platform that centralizes the core processes of a school while respecting the rights and responsibilities of each role?

Project Objectives

Objective	Description
Centralization	Group school information into one coherent reference system.
Role-based access	Adapt features to the responsibilities of the administrator, receptionist, accountant, teacher, student, and parent.
Automation	Reduce manual work related to enrollments, attendance, grades, payments, and communication.
Traceability	Keep fa clear record of important operations and make follow-up easier.
Scalability	Build a modular architecture that can integrate future modules.

Work Methodology

Phase	Work Performed
Preliminary study	Understanding the school domain, stakeholders, existing documents, and limits of manual management.
Requirements analysis	Identifying actors, functional requirements, non-functional requirements, and use cases.
Design	Defining the architecture, UML models, database model, REST API structure, and user-interface organization.
Implementation	Developing frontend interfaces, JavaScript services, FastAPI routers, and data managers.
Validation	Checking business scenarios, API behavior, role navigation, and interface consistency.

Report Organization

This report is organized into four chapters. Chapter 1 presents the existing-system study and the proposed solution. Chapter 2 formalizes the requirements and use cases. Chapter 3 explains the system design, including architecture, UML models, database structure, API design, and security. Chapter 4 presents the implementation, application interfaces, and validation scenarios. A general conclusion summarizes the work and proposes future improvements.

Chapter 1 - Existing-System Study and Problem Statement

Introduction

Before designing a software solution, it is necessary to understand how a school operates and where current practices fail. This chapter presents the domain, the main stakeholders, the business processes, the limitations of traditional management, and the proposed solution.

Domain Overview

A school manages administrative, academic, financial, and communication activities. These activities are connected: student enrollment affects classes, schedules, resources, attendance, grades, and fees. A global view is therefore required to avoid disconnected processes between departments.

Process	Managed Data	Stakeholders
Student enrollment and file	Identity, parent, class, program, and status	Administration, reception, parent
Academic follow-up	Subjects, assignments, assessments, grades, and attendance	Teacher, student, parent, administration
Financial management	Fees, payments, transactions, and reminders	Accountant, reception, parent, administration
Communication	Messages, conversations, notifications, and posts	All profiles
Administrative control	Users, roles, classes, programs, and resources	Administrator

Existing-System Study

In many schools, management remains partially manual: paper registers, Excel files, uncentralized conversations, and documents kept separately by each service. These tools are easy to start with, but they become fragile when the number of students, teachers, and daily operations increases.

Existing Solution	Advantages	Limitations
Paper registers	Simple and independent from technical infrastructure	Slow search, risk of loss, duplication, and no reliable backup
Excel files	Flexible and quick for simple lists	Possible inconsistencies, difficult sharing, and weak access control
Informal messaging	Fast communication between people	Scattered information, weak traceability, and no business context
Disconnected applications	Automate some isolated tasks	Limited consistency between academic, financial, and communication modules

Critique of the Existing System

The study reveals recurring issues: redundant data entry, slow information retrieval, difficult consolidation of grades and attendance, possible financial errors, lack of role-based dashboards, and weak communication follow-up. These limits

reduce administrative responsiveness and the quality of academic supervision.

Observed Limit	Consequence	Expected Response
Information dispersion	Decisions may rely on incomplete data	Centralized reference system
Repeated manual entry	Errors and time loss	Controlled forms and shared APIs
Undifferentiated access	Risk of unauthorized consultation or modification	Role-based dashboards and permissions
Unstructured communication	Lost messages and weak traceability	Integrated messages and notifications
Fragile financial follow-up	Delays, duplicates, and incorrect balances	Fees, payments, and transactions linked to students

Problem Statement

The project problem can be stated as follows: how can we build a web application that centralizes the main processes of a school, provides each stakeholder with a dedicated interface, and ensures reliable information flow between administration, teachers, students, parents, and the finance office?

Proposed Solution

The proposed solution is a role-based web platform called School Management System. It clearly separates frontend, backend API, and database responsibilities. Each role has a specific dashboard, while the data remains synchronized through common REST services.

Solution Area	Contribution
Functional coverage	Users, students, parents, teachers, classes, enrollments, attendance, assessments, grades, resources, fees, payments, messages, notifications, and posts.
Maintainable architecture	Static frontend organized by roles, modular FastAPI backend, data managers, and MySQL database.
Role-based approach	Each profile accesses the features that match its responsibilities.
Integrated communication	Messages, conversations, notifications, and posts reduce scattered exchanges.
Internationalization readiness	Translation files prepare the interface for multiple languages.

Conclusion

The existing-system study confirms the need for a centralized, structured, and extensible platform. The next chapter transforms this observation into functional requirements, non-functional requirements, and use cases.

Chapter 2 - Requirements Analysis

Introduction

Requirements analysis transforms the problem statement into concrete specifications. It identifies the actors, expected functions, quality constraints, and use cases that guide the UML design and the implementation.

Actor Identification

Actor	Main Responsibilities	Application Space
Administrator	Manage users, academic data, resources, finance, posts, and notifications.	Admin dashboard
Receptionist	Create and consult student/parent files, search records, and perform front-office operations.	Reception dashboard
Accountant	Track fees, payments, transactions, balances, and financial notifications.	Finance dashboard
Teacher	Consult assignments, manage attendance, assessments, grades, resources, and academic communication.	Teacher dashboard
Student	Consult schedule, grades, attendance, resources, fees, messages, and notifications.	Student dashboard
Parent	Follow linked children: grades, attendance, fees, publications, messages, and notifications.	Parent dashboard

Functional Requirements

Module	Expected Features
Authentication	Login, role identification, dashboard redirection, and logout.
People management	Create and update users, students, parents, and teachers with active/inactive states.
Academic management	Classes, programs, enrollments, teacher assignments, schedules, attendance, assessments, and grades.
Resources	Add, consult, and download educational resources associated with classes or subjects.
Finance	Student fees, payments, balances, user transactions, overdue follow-up, and financial notifications.
Communication	Conversations, messages, targeted notifications, and posts visible from the interfaces.
Preferences	Theme, language, and visual experience management by role.

Non-Functional Requirements

Requirement	Description
Security	Role-based access, input validation, protection of sensitive operations, and controlled error handling.
Performance	Fast table loading, targeted API requests, and separated responsibilities to reduce coupling.

Requirement	Description
Usability	Readable interfaces, section-based navigation, user feedback, and responsive behavior.
Maintainability	Organization by frontend modules, backend routers, and reusable data managers.
Reliability	Relational database consistency, foreign-key constraints, and API-side error handling.
Scalability	Ability to add modules, roles, languages, or reports without rewriting the whole system.

Use Case Diagrams

The following diagrams define the functional boundary of the system for each profile. They help verify that responsibilities are separated and that each actor has a coherent functional scope.

Use Case Relationship Notation

Notation	Meaning	Use in this report
Association	A solid line links an actor to the main use cases that the actor can initiate.	Used between each role and its main dashboard, academic, financial, or communication functions.
<>	A dashed arrow from a base use case to a required sub-use case.	Used when a function systematically needs another function, such as authentication before dashboard access.
<>	A dashed arrow from an optional or conditional use case to the use case it extends.	Used for optional behavior such as language/theme changes or contextual notifications.
System boundary	The rectangle represents the application scope.	All use cases inside the boundary belong to the School Management System.

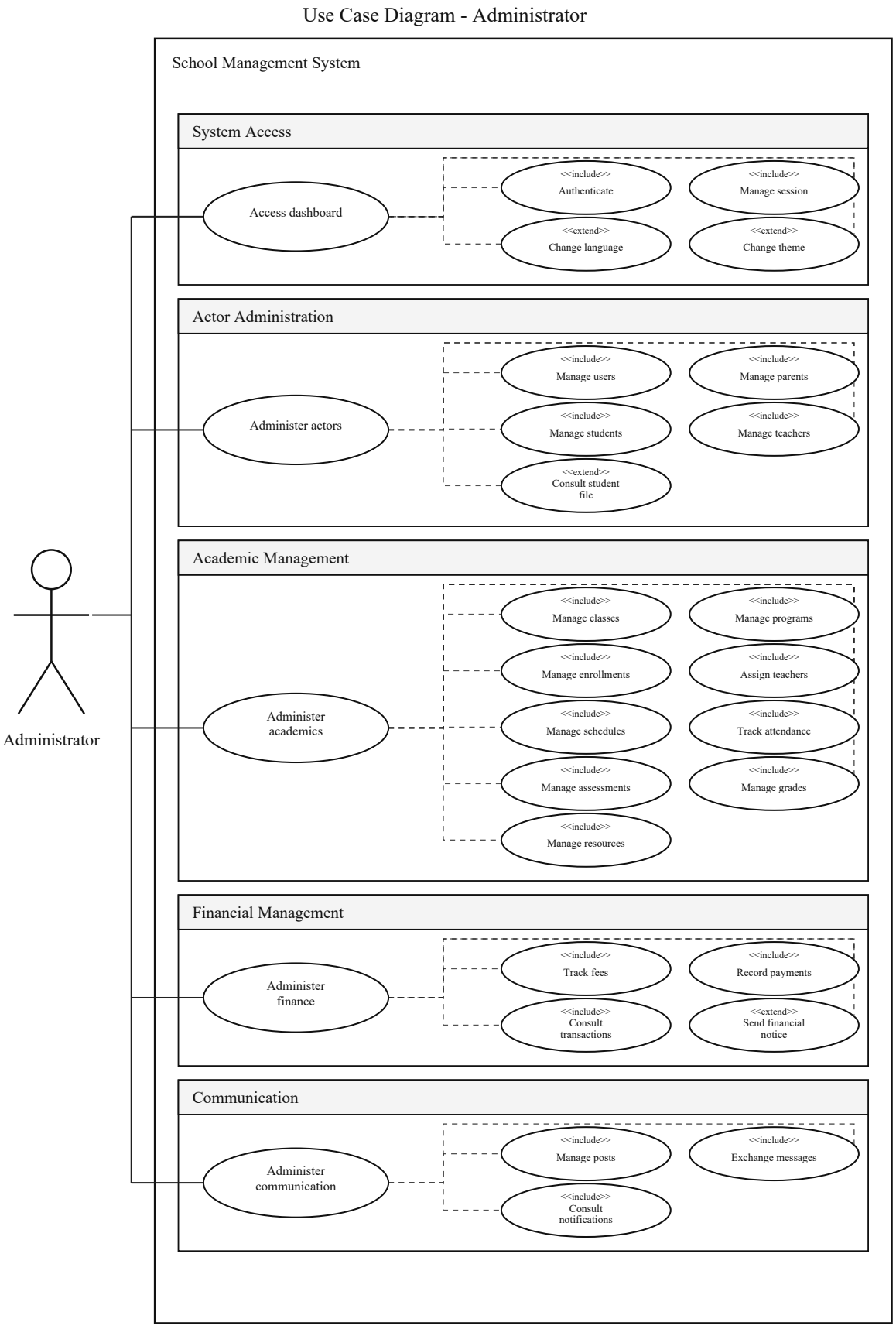

Figure 2.1 - Use case diagram: Administrator.

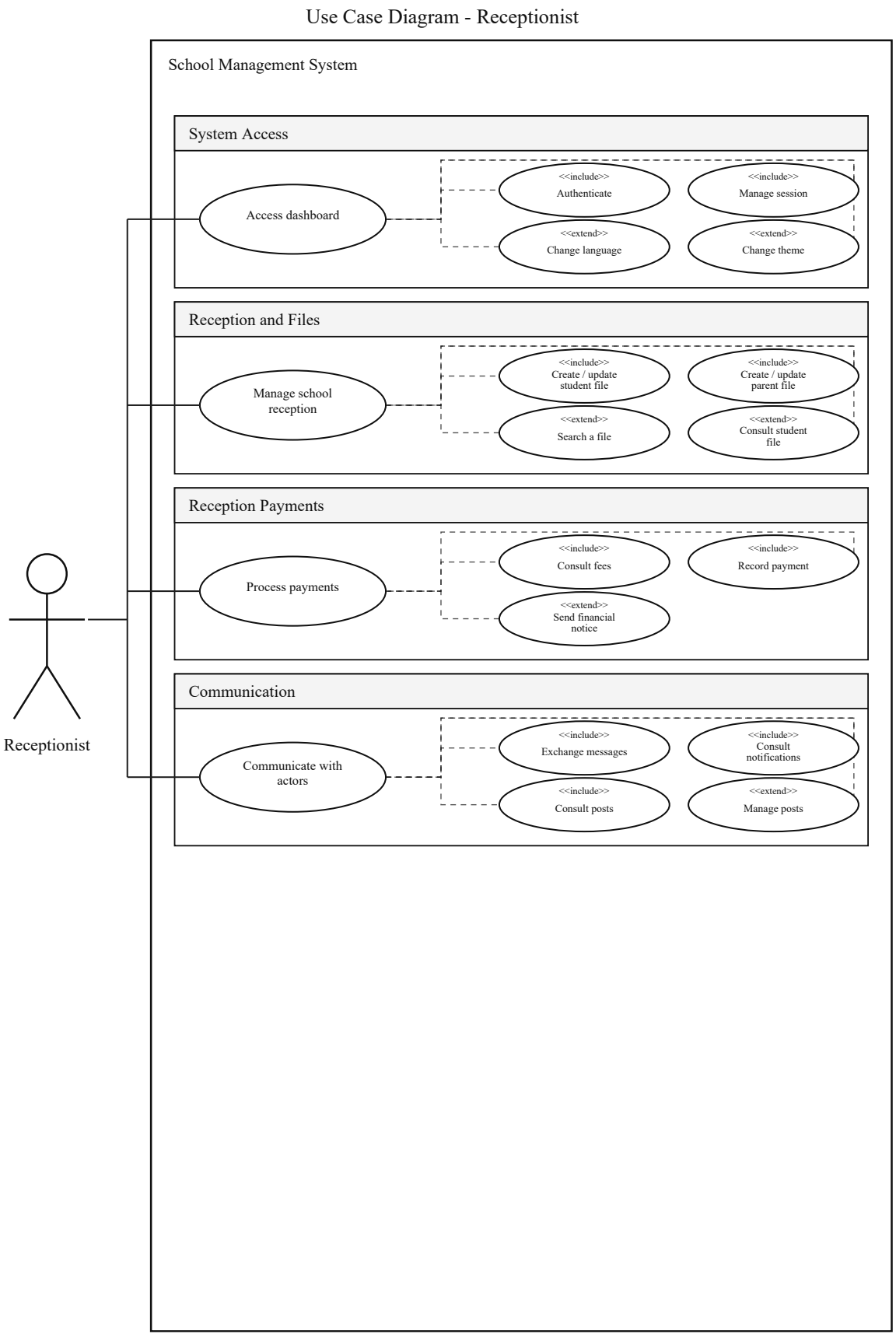

Figure 2.2 - Use case diagram: Receptionist.

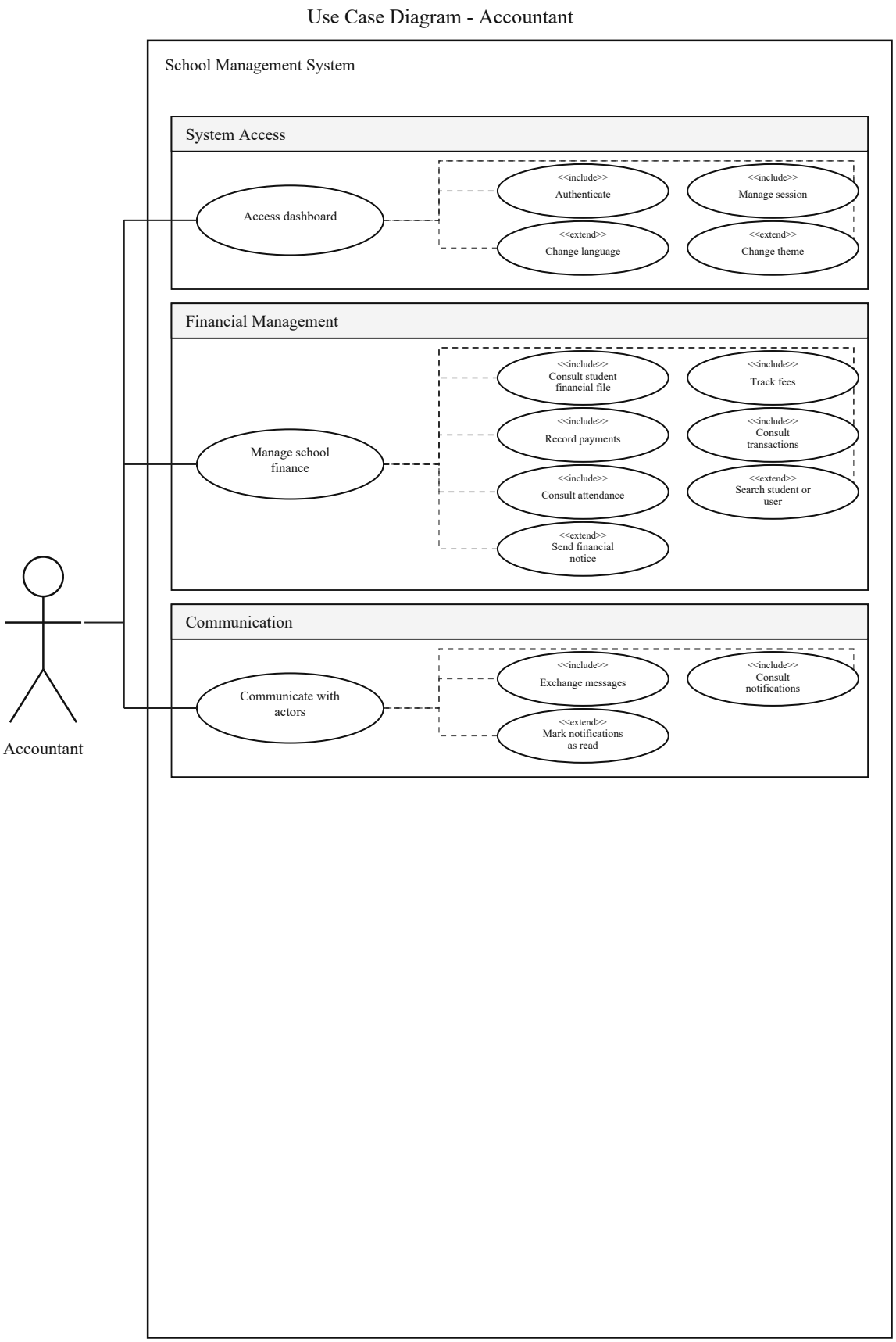

Figure 2.3 - Use case diagram: Accountant.

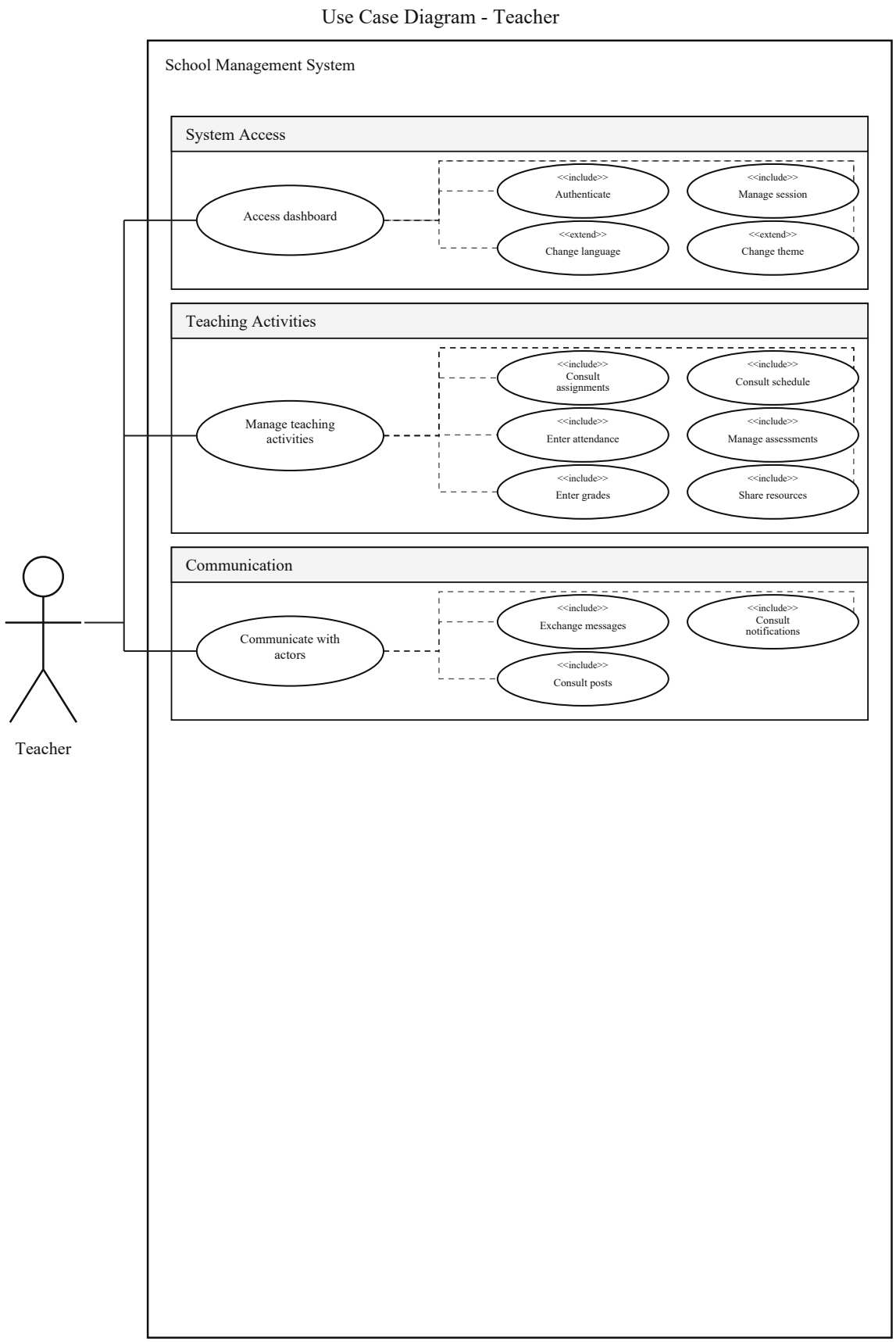

Figure 2.4 - Use case diagram: Teacher.

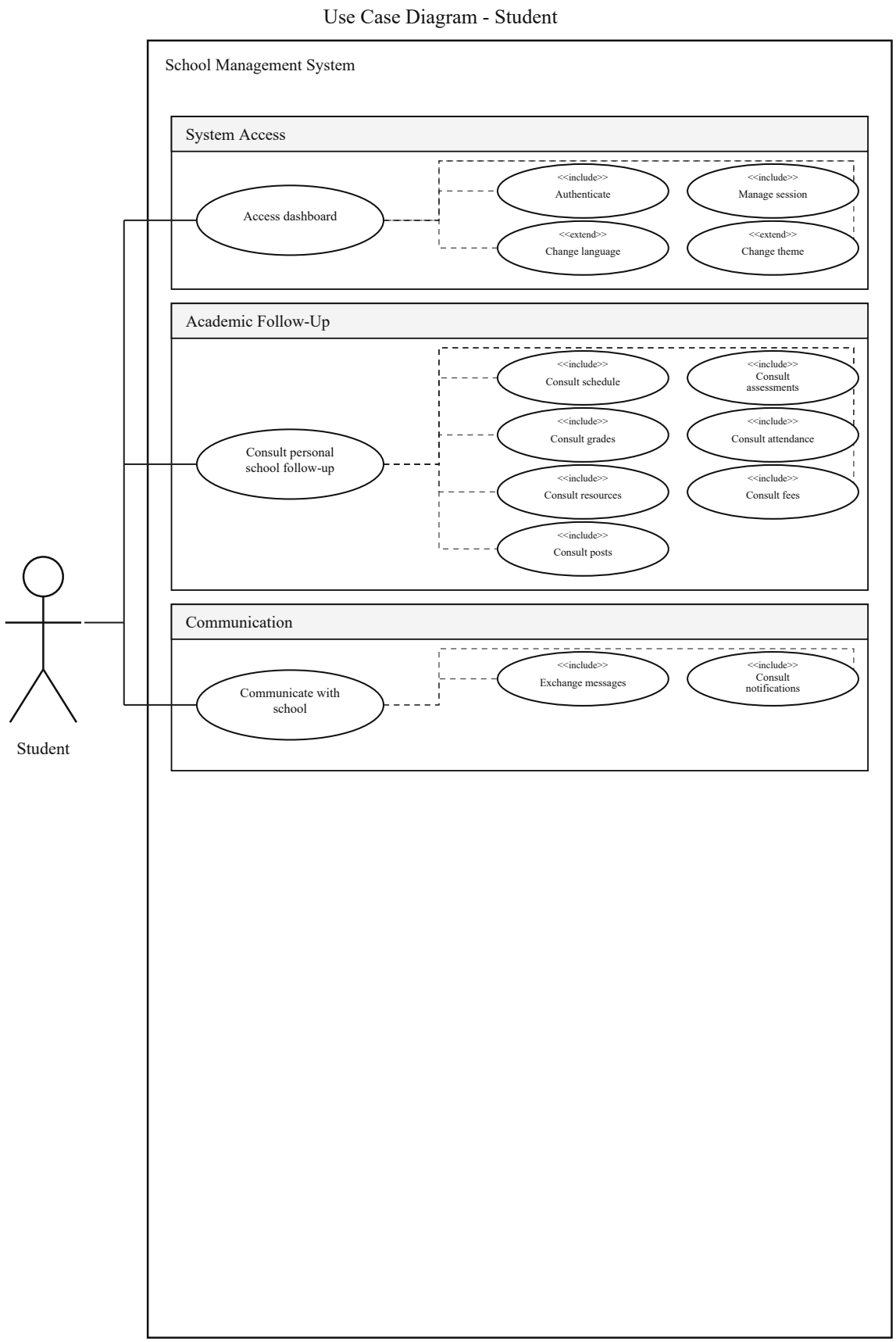

Figure 2.5 - Use case diagram: Student.

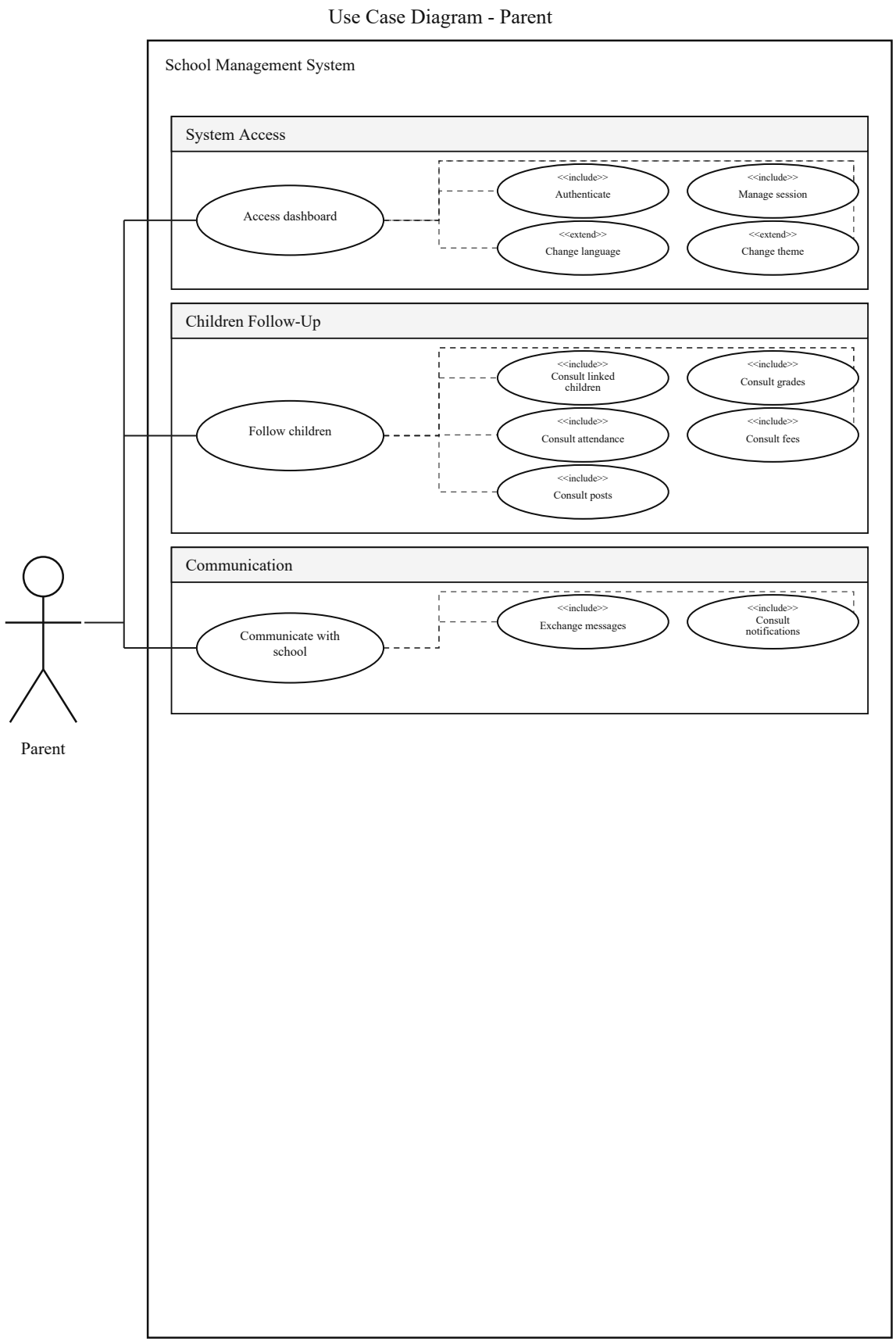

Figure 2.6 - Use case diagram: Parent.

Textual Description of Main Use Cases

Use Case	Main Actor	Description
Log in	All profiles	The user enters credentials, the backend verifies the account, and the frontend opens the dashboard that matches the role.
Manage users	Administrator	The administrator creates, updates, activates, or deactivates accounts and assigns application roles.
Enroll a student	Administrator / Receptionist	A student file is created, linked to a parent, assigned to a class, and made available for academic and financial follow-up.
Record attendance	Teacher	The teacher opens the class sheet, marks attendance or absence, and records justifications when required.
Enter grades	Teacher	The teacher selects an assessment, enters grades, and makes results available to the student and parent.
Record a payment	Accountant	The accountant selects a financial file, enters the amount, updates the balance, and keeps the operation history.
Exchange a message	All profiles	A user opens a conversation, sends a message, and receives new exchanges through the internal messaging module.

Conclusion

The identified requirements confirm the need for a multi-module application unified by the same authentication, navigation, and data logic. The next chapter details the technical design that satisfies these requirements.

Chapter 3 - System Design

Introduction

The design chapter explains how requirements are transformed into software architecture, data models, UML diagrams, REST APIs, and user interfaces. The goal is to ensure a coherent, maintainable, and extensible structure.

General System Architecture

The system follows a separated-layer web architecture. The frontend manages display, user experience, and role interactions. The FastAPI backend exposes business services through REST endpoints. The MySQL database stores data, relationships, and integrity constraints.

Layer	Responsibility	Project Elements
Presentation	Display dashboards, forms, tables, messages, and preferences.	HTML, CSS, JavaScript, role pages, i18n, auth, api, and preferences modules.
API services	Receive requests, validate data, apply business logic, and return JSON responses.	FastAPI, routers, dependencies, Pydantic, and Uvicorn.
Data	Store entities, relationships, history, and integrity constraints.	MySQL, data managers, relational tables, and foreign keys.

Nominal Data Flow

A typical scenario starts with a user action in the dashboard. The frontend controller calls a JavaScript service, which sends an HTTP request to the appropriate FastAPI router. The router delegates the operation to a data manager and returns a structured response to the frontend. The interface is then updated without exposing the database directly to the client.

Database Modeling

The database is organized around school and relational entities. The tables cover identity, profiles, classes, programs, enrollments, assignments, resources, assessments, grades, attendance, finance, and communication.

View	Description
Conceptual model	The main concepts are User, Student, Parent, Teacher, Class, Program, Enrollment, Assessment, Grade, Attendance, Fee, Payment, Message, Notification, and Post.
Logical model	Relationships are translated into tables with primary and foreign keys. A parent can be linked to several students, a teacher receives assignments, a class groups enrollments, and an assessment produces grades.
Physical model	The MySQL implementation defines data types, constraints, indexes, and integrity rules required by the API.

Family	Main Tables
Identity	roles, users, parents, students, teachers
Academic	programs, classes, student_enrollments, teacher_assignments, subjects, resources, assessments, grades, attendance, schedules
Finance	student_fees, payments, user_transactions

Family	Main Tables
Communication	conversations, messages, notifications, posts

Class Diagram

The class diagram summarizes the backend structure and the main relationships between domain entities and data managers. It is a reference for understanding how the system modules cooperate around the same database.

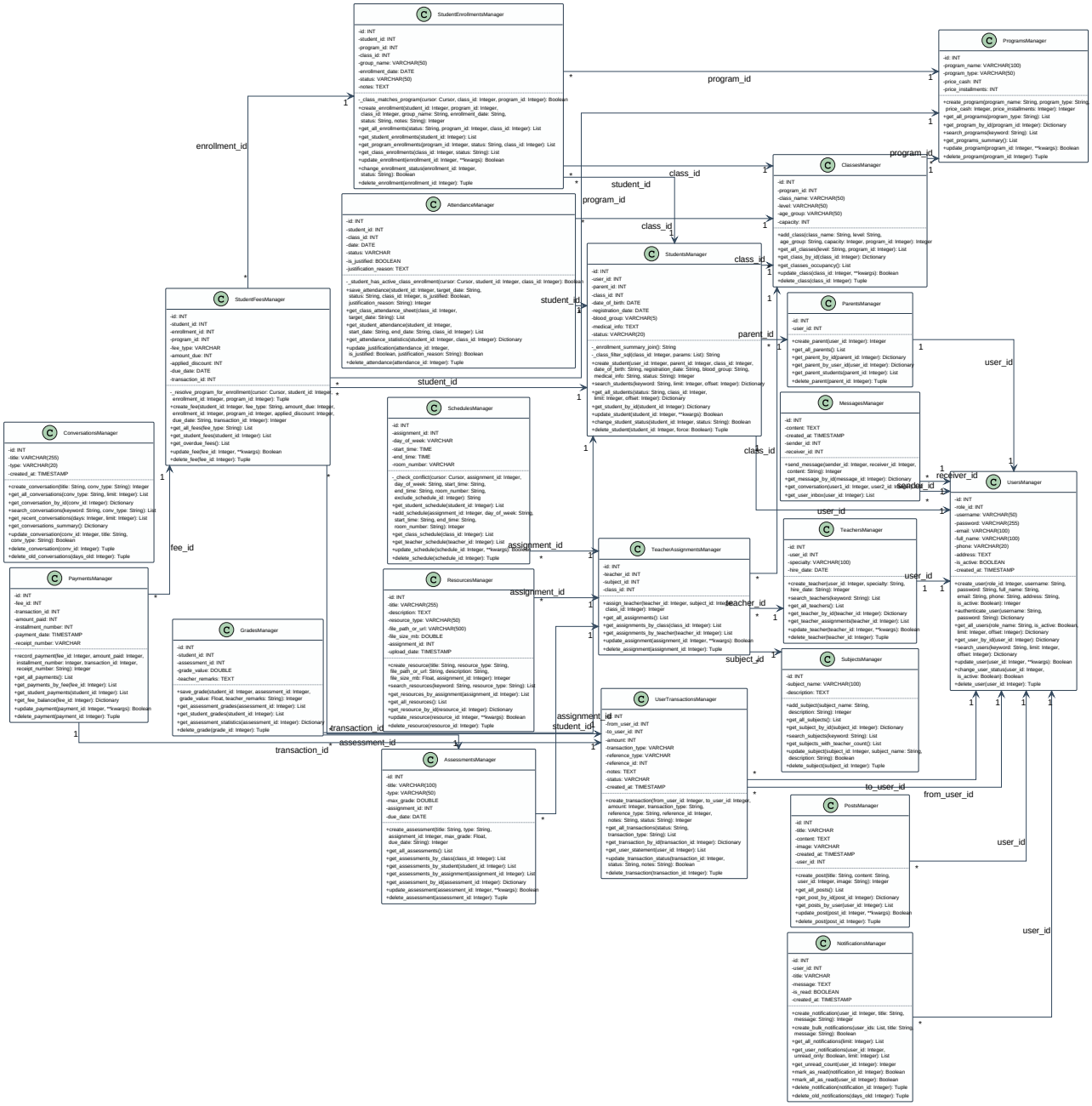

Figure 3.1 - School Management System class diagram.

Sequence Diagrams

Sequence diagrams describe the behavior of the system at runtime. They show collaboration between the user, dashboard, frontend controller, API service, FastAPI router, and data manager.

Selected Dynamic Scenarios

Role	Represented Scenario	Design Purpose
Administrator	Dashboard loading, consultation, and global CRUD operations.	Validate central control and calls to the main modules.
Receptionist	File creation, file consultation, and front-office search.	Show the relationship between reception, students, parents, and payments.
Accountant	Fee follow-up, payments, transactions, and notifications.	Separate financial operations from general administration.
Teacher	Assignments, attendance, assessments, grades, resources, and messages.	Formalize academic flows from classroom work to family follow-up.
Student	Consultation of personal and academic information.	Ensure secure read access to data linked to the connected profile.
Parent	Aggregation of information for linked children.	Illustrate multi-child follow-up and communication with the school.

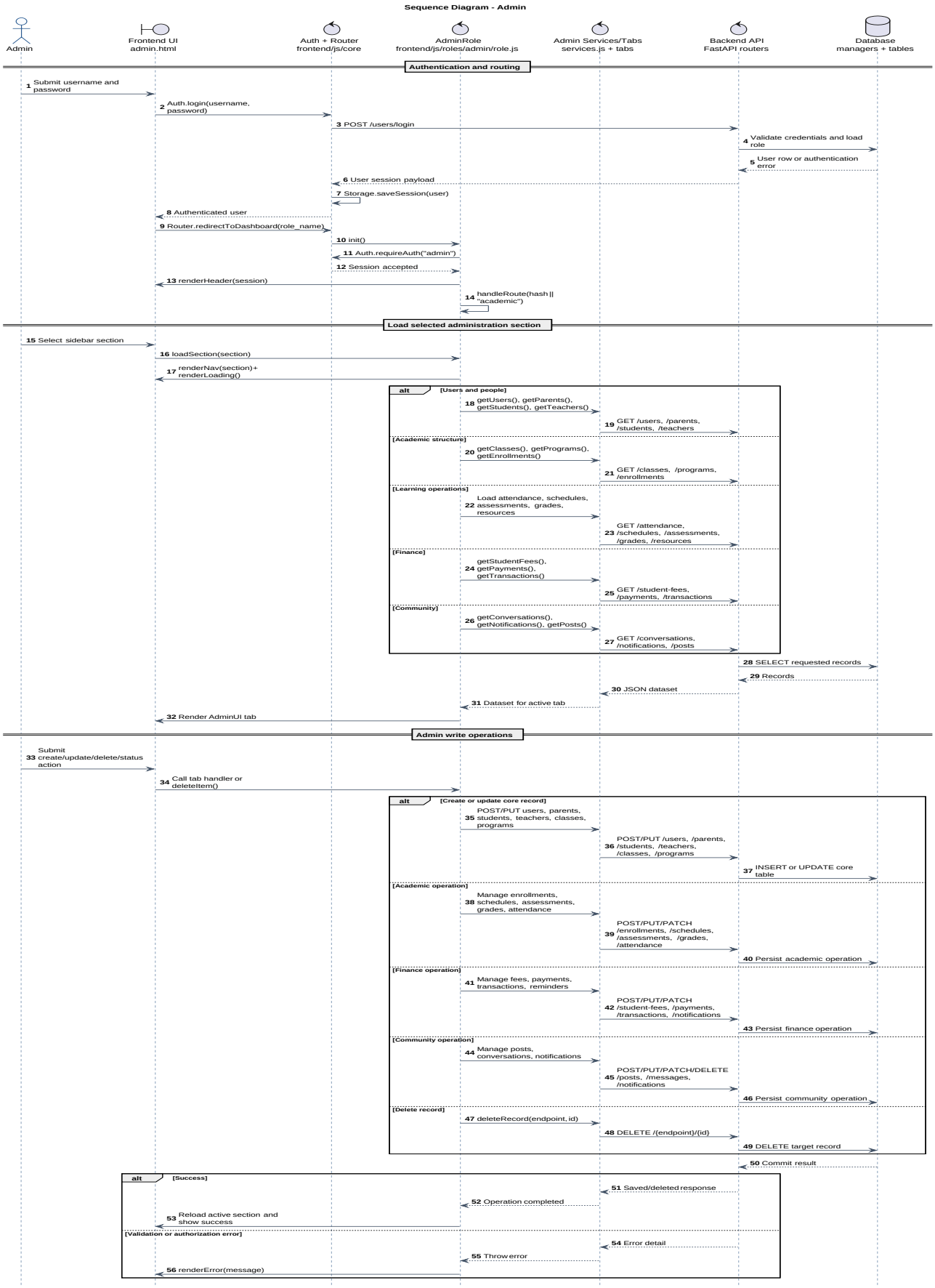

Figure 3.2 - Sequence diagram: Administrator.

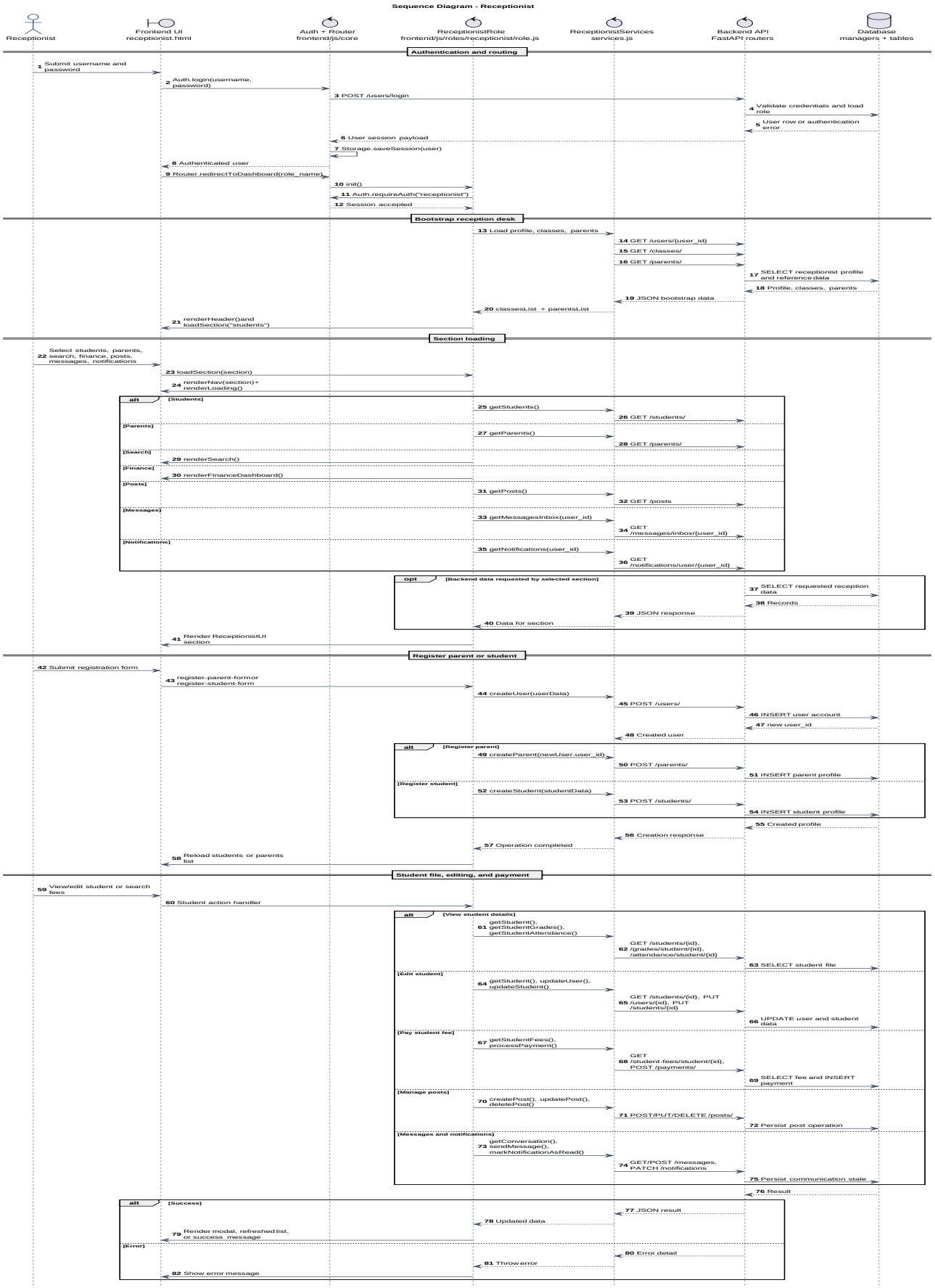

Figure 3.3 - Sequence diagram: Receptionist.

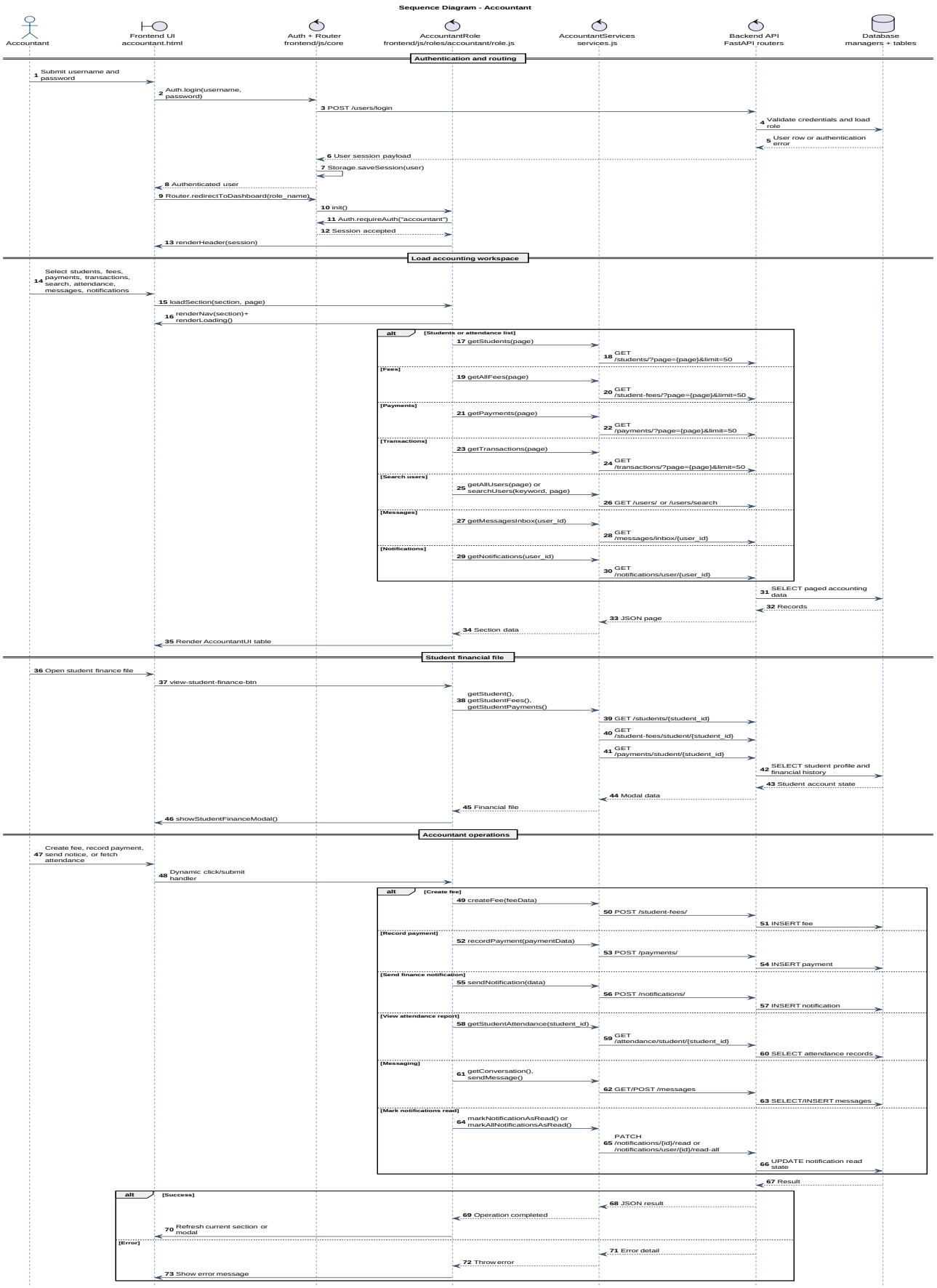

Figure 3.4 - Sequence diagram: Accountant.

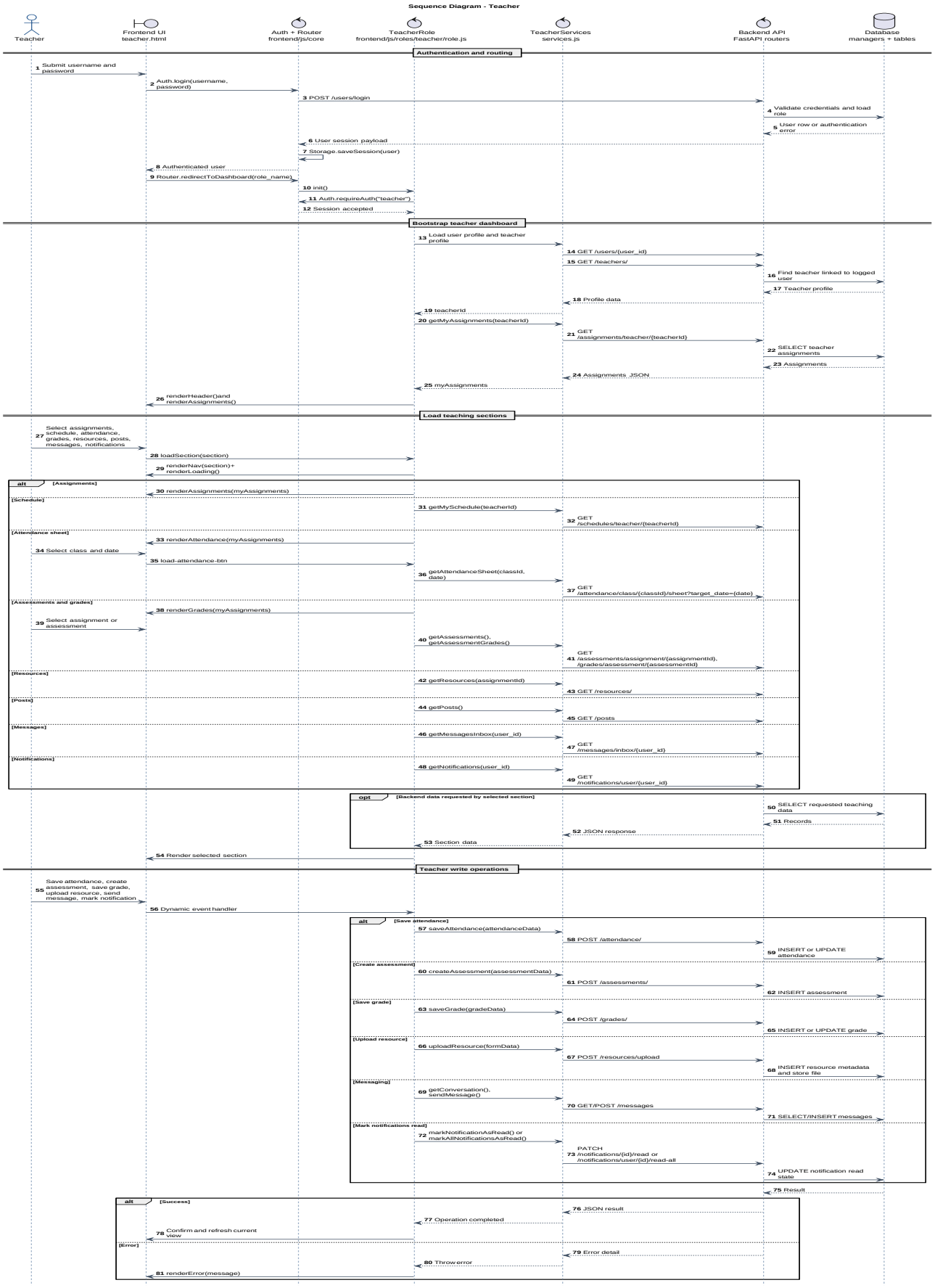

Figure 3.5 - Sequence diagram: Teacher.

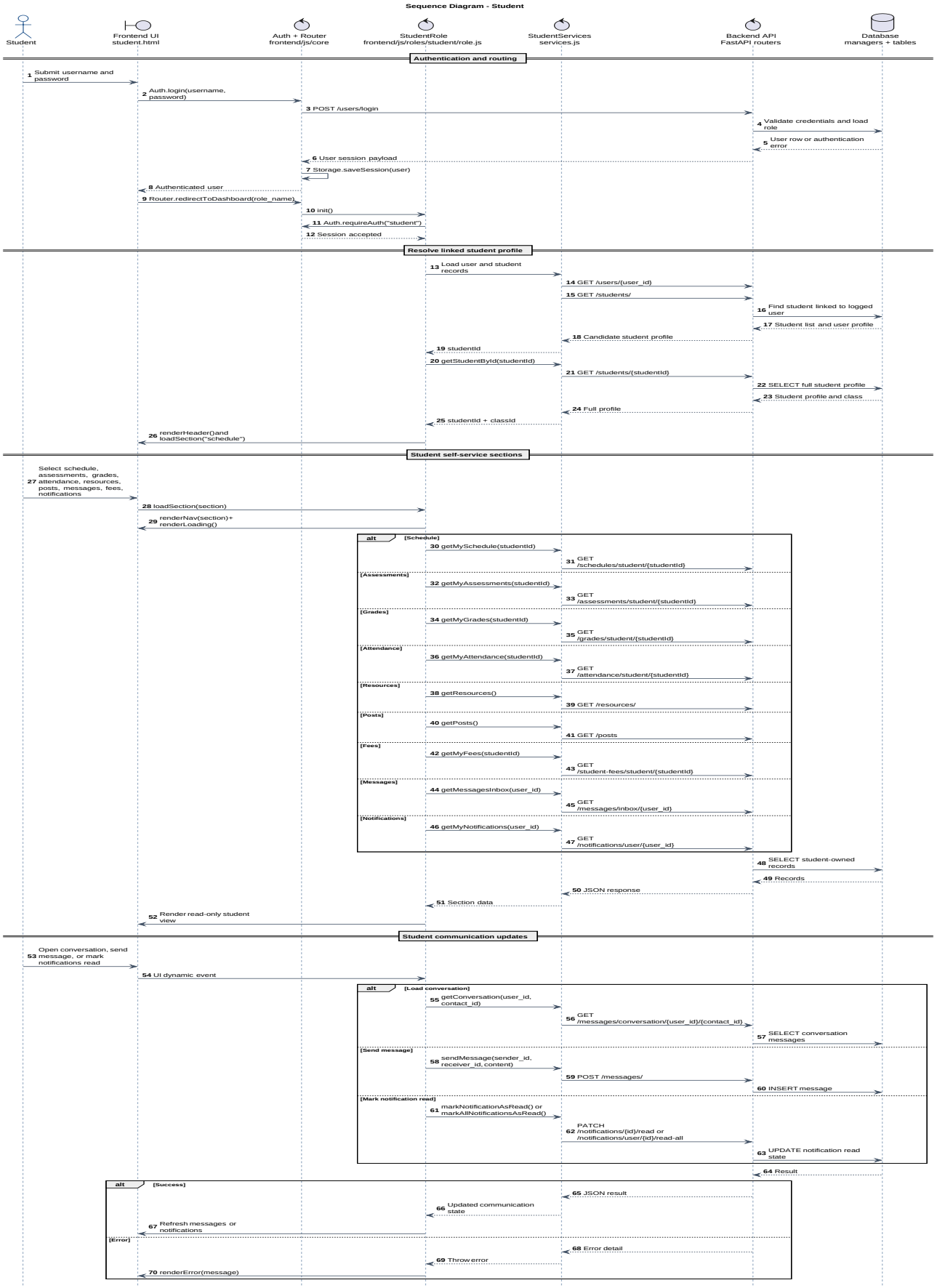

Figure 3.6 - Sequence diagram: Student.

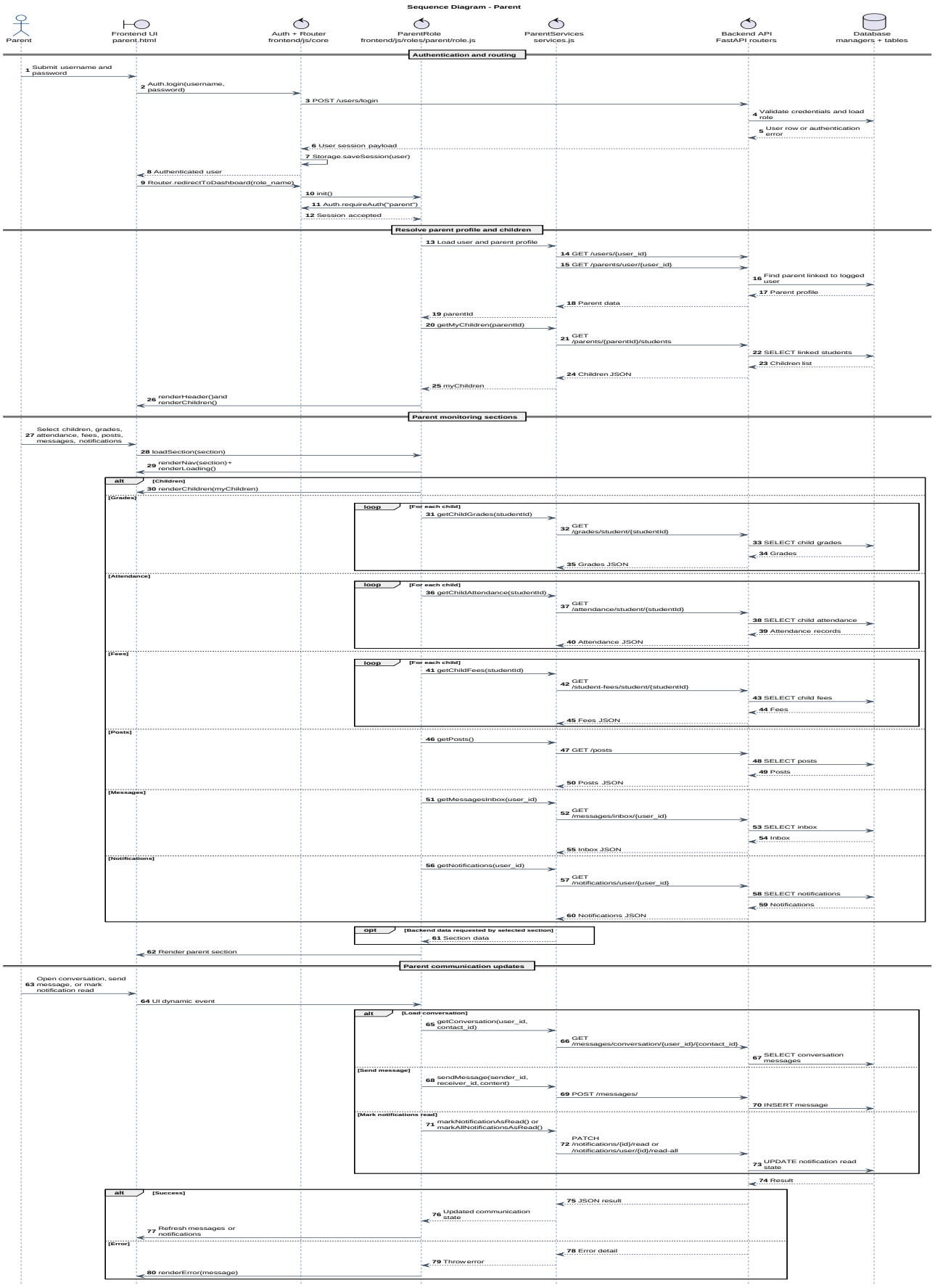

Figure 3.7 - Sequence diagram: Parent.

User Interface Design

The interface follows a simple principle: one common entry point, authentication, and then a workspace dedicated to the connected role. This separation reduces visual complexity and avoids mixing features that do not belong to the same actor.

Design Principles

Principle	Application in the Project
Clarity	Each dashboard groups modules into readable and directly accessible sections.
Consistency	Roles share common components: authentication, API client, local storage, language, theme, messages, and notifications.
Role separation	Admin, receptionist, accountant, teacher, student, and parent pages have their own controllers and services.
Language readiness	Translation files make the interface ready for multiple languages.
Visual continuity	Theme and language preferences follow the user across navigation.

Page Description

Page	Main Content
Login	Authentication, language selection, theme selection, and public posts before login.
Admin	Users, students, parents, teachers, classes, programs, finance, resources, posts, messages, and notifications.
Receptionist	Student and parent files, search, front-office operations, financial consultation, posts, messages, and notifications.
Accountant	Students, fees, payments, transactions, search, attendance, and financial notifications.
Teacher	Assignments, schedule, attendance, assessments, grades, resources, posts, and messaging.
Student	Schedule, assessments, grades, attendance, resources, fees, posts, and messages.
Parent	Child follow-up, grades, attendance, fees, posts, messages, and notifications.

Navigation Diagram

The navigation diagram presents the transition from the entry page to specialized dashboards. It also shows the shared controls reused across the different spaces.

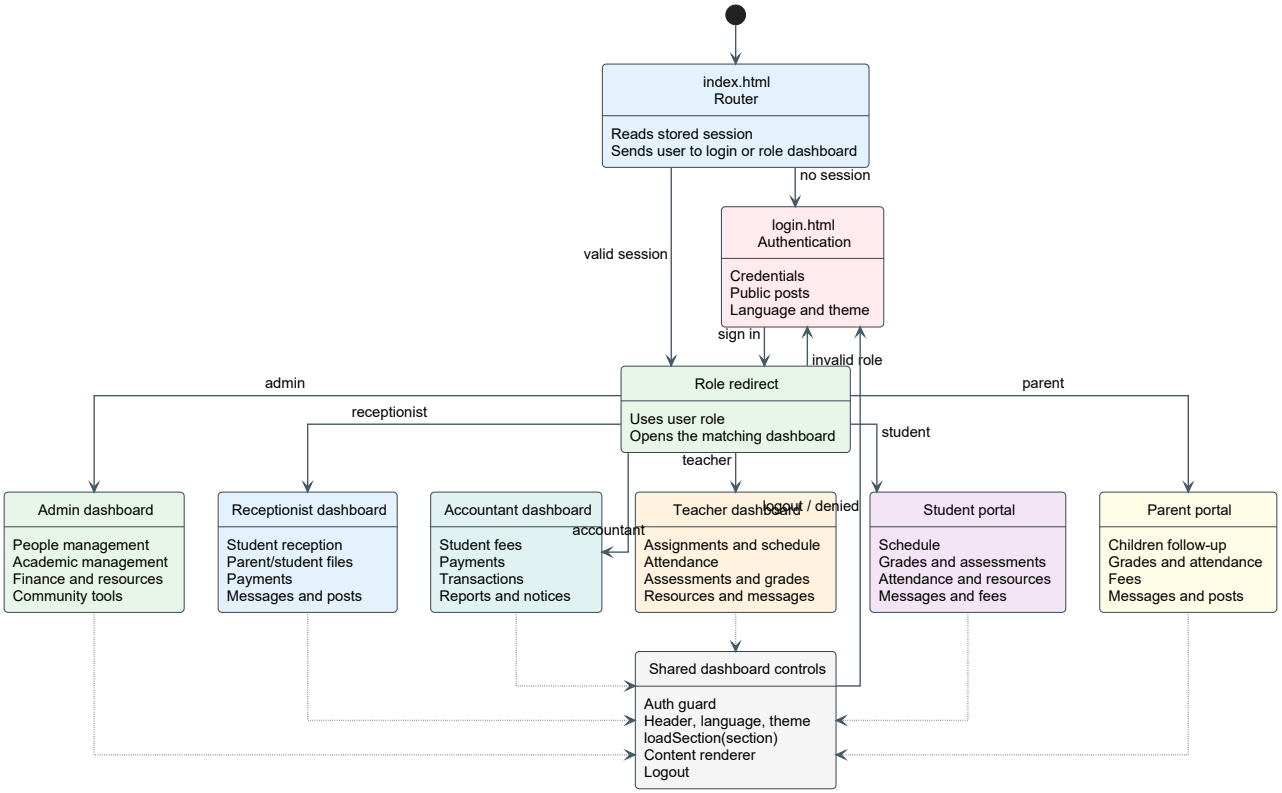

Figure 3.8 - Role-based navigation diagram.

Technical Choices

The technical choices were selected to keep the application simple to deploy, understandable for a bachelor's degree project, and modular enough to evolve. The system relies on standard web technologies, a clear REST API, and a relational database.

Area	Technologies	Justification
Frontend	HTML5, CSS3, vanilla JavaScript	Provide a lightweight interface organized by pages, roles, services, and UI components.
Backend	Python, FastAPI, Pydantic, Uvicorn	Expose readable, validated, and easy-to-document REST endpoints.
Database	MySQL, mysql-connector-python, SQLAlchemy utilities	Store relational data with constraints and business relationships.
Documentation	PlantUML, SVG, ReportLab, svglib	Produce versionable diagrams and a reproducible PDF report.
Internationalization	JSON translation files	Prepare the interface for multiple languages.

REST API Design

The backend exposes specialized routers. Each router represents a functional domain and uses a data manager responsible for SQL operations. This organization makes endpoints easier to test, document, and extend.

API Family	Representative Endpoints	Role
Users	/users, /users/login, /users/search	Authentication, accounts, search, and user states.
School structure	/students, /parents, /teachers, /classes, /programs, /enrollments, /assignments	Profile management and academic organization.
Learning	/attendance, /schedules, /assessments, /grades, /resources	Daily follow-up, assessments, results, and educational resources.
Finance	/student-fees, /payments, /transactions	Fees, payments, balances, and movements.
Communication	/conversations, /messages, /notifications, /posts	Internal exchanges, alerts, and publications.

Backend Design Structure

The backend follows a layered organization. The FastAPI application loads routers, routers validate HTTP requests and delegate business operations, and database managers isolate SQL access from the API layer.

Backend Element	Responsibility
backend/app.py	Creates the FastAPI application, enables CORS, initializes the database, and includes all routers.
backend/apis/*_api.py	Defines REST and WebSocket endpoints by functional domain: users, students, finance, resources, communication, and academic follow-up.
backend/apis/dependencies.py	Provides manager instances through FastAPI dependency injection.
backend/database/*_manager.py	Contains data-access operations and domain-specific business queries.

Backend Element	Responsibility
backend/database/base/*	Centralizes configuration, connection handling, schema creation, tables, views, and indexes.
backend/uploads/resources	Stores uploaded learning resources referenced by the resources API.
backend/requirements.txt	Lists Python dependencies required to run the backend service.

Backend Request Flow

Step	Description
1. Frontend service call	A role-specific JavaScript service sends a request to the REST API.
2. FastAPI router	The router receives the request, validates parameters and request bodies, and selects the correct operation.
3. Dependency injection	FastAPI injects the corresponding database manager from the dependencies module.
4. Data manager	The manager executes SQL operations and enforces domain behavior.
5. Database	MySQL stores users, school entities, academic records, finance data, and communication content.
6. Response handling	The API returns JSON or a file response, then the frontend updates the interface.

Frontend Design Structure

The frontend is implemented as a structured static web application. Common services are shared in core modules, while role folders contain the behavior and user-interface logic specific to each dashboard.

Frontend Element	Responsibility
frontend/index.html	Entry point used to redirect users to the appropriate login or dashboard page.
frontend/pages/*.html	Role-based pages for administrator, receptionist, accountant, teacher, student, parent, and login.
frontend/js/core	Shared services for API calls, authentication, routing, preferences, storage, and internationalization.
frontend/js/roles/	Role-specific controllers, services, UI logic, and tab modules.
frontend/css	Global and role-specific style sheets for dashboards and interface sections.
frontend/locales/	Translation dictionaries used by the internationalization layer.
frontend/assets	Visual assets such as the logo and favicon.
frontend/server_http.py	Small local HTTP server used to serve the static frontend during development or demonstration.

API Endpoint Reference

The following table is generated from the backend router files. It documents the operational surface exposed by the system and provides a technical reference for implementation, testing, and maintenance.

Domain	Method	Endpoint	Purpose
Root	GET	/	Read root API health message
Assessments	POST	/assessments	Create Assessment
Assessments	GET	/assessments	Get All Assessments
Assessments	GET	/assessments/class/{class_id}	Get Assessments By Class
Assessments	GET	/assessments/student/{student_id}	Get Assessments By Student
Assessments	GET	/assessments/assignment/{assignment_id}	Get Assessments By Assignment
Assessments	GET	/assessments/{assessment_id}	Get Assessment
Assessments	PUT	/assessments/{assessment_id}	Update Assessment
Assessments	DELETE	/assessments/{assessment_id}	Delete Assessment
Attendance	POST	/attendance	Save Attendance
Attendance	GET	/attendance/class/{class_id}/sheet	Get Class Attendance Sheet
Attendance	GET	/attendance/student/{student_id}	Get Student Attendance
Attendance	GET	/attendance/student/{student_id}/statistics	Get Attendance Statistics
Attendance	PATCH	/attendance/{attendance_id}/justification	Update Justification
Attendance	DELETE	/attendance/{attendance_id}	Delete Attendance
Classes	POST	/classes	Add Class
Classes	GET	/classes/occupancy	Get Classes Occupancy
Classes	GET	/classes	Get All Classes
Classes	GET	/classes/{class_id}	Get Class
Classes	PUT	/classes/{class_id}	Update Class
Classes	DELETE	/classes/{class_id}	Delete Class
Conversations	POST	/conversations	Create Conversation
Conversations	GET	/conversations/summary	Get Summary
Conversations	GET	/conversations/search	Search Conversations
Conversations	GET	/conversations/recent	Get Recent Conversations
Conversations	DELETE	/conversations/old	Delete Old Conversations
Conversations	GET	/conversations	Get Conversations
Conversations	GET	/conversations/{conv_id}	Get Conversation
Conversations	PUT	/conversations/{conv_id}	Update Conversation

Domain	Method	Endpoint	Purpose
Conversations	DELETE	/conversations/{conv_id}	Delete Conversation
Grades	POST	/grades	Save Grade
Grades	GET	/grades/assessment/{assessment_id}	Get Assessment Grades
Grades	GET	/grades/student/{student_id}	Get Student Grades
Grades	GET	/grades/assessment/{assessment_id}/statistics	Get Assessment Statistics
Grades	DELETE	/grades/{grade_id}	Delete Grade
Messages	WS	/messages/ws/{user_id}	Websocket Endpoint
Messages	POST	/messages	Send Message
Messages	GET	/messages/conversation/{user1_id}/{user2_id}	Get Conversation
Messages	GET	/messages/inbox/{user_id}	Get Inbox
Messages	PUT	/messages/{message_id}	Update Message
Messages	DELETE	/messages/{message_id}	Delete Message
Notifications	GET	/notifications	Get All Notifications
Notifications	POST	/notifications	Create Notification
Notifications	POST	/notifications/bulk	Create Bulk Notifications
Notifications	GET	/notifications/user/{user_id}/unread-count	Get Unread Count
Notifications	GET	/notifications/user/{user_id}	Get User Notifications
Notifications	PATCH	/notifications/{notification_id}/read	Mark As Read
Notifications	PATCH	/notifications/user/{user_id}/read-all	Mark All As Read
Notifications	DELETE	/notifications/old	Delete Old Notifications
Notifications	DELETE	/notifications/{notification_id}	Delete Notification
Parents	POST	/parents	Create Parent
Parents	GET	/parents	Get Parents
Parents	GET	/parents/{parent_id}	Get Parent
Parents	GET	/parents/user/{user_id}	Get Parent By User
Parents	GET	/parents/{parent_id}/students	Get Parent Students
Parents	DELETE	/parents/{parent_id}	Delete Parent
Payments	POST	/payments	Record Payment

Domain	Method	Endpoint	Purpose
Payments	GET	/payments	Get All Payments
Payments	GET	/payments/fee/{fee_id}	Get Payments By Fee
Payments	GET	/payments/student/{student_id}	Get Student Payments
Payments	GET	/payments/fee/{fee_id}/balance	Get Fee Balance
Payments	PUT	/payments/{payment_id}	Update Payment
Payments	DELETE	/payments/{payment_id}	Delete Payment
Posts	POST	/posts	Create Post
Posts	GET	/posts	Get All Posts
Posts	GET	/posts/user/{user_id}	Get Posts By User
Posts	GET	/posts/{post_id}	Get Post
Posts	PUT	/posts/{post_id}	Update Post
Posts	DELETE	/posts/{post_id}	Delete Post
Programs	POST	/programs	Create Program
Programs	GET	/programs/summary	Get Programs Summary
Programs	GET	/programs/search	Search Programs
Programs	GET	/programs	Get All Programs
Programs	GET	/programs/{program_id}	Get Program
Programs	PUT	/programs/{program_id}	Update Program
Programs	DELETE	/programs/{program_id}	Delete Program
Resources	POST	/resources/upload	Upload Resource
Resources	POST	/resources	Create Resource Link
Resources	GET	/resources	Get All Resources
Resources	GET	/resources/{resource_id}	Get Resource
Resources	GET	/resources/{resource_id}/download	Download Resource
Resources	PUT	/resources/{resource_id}	Update Resource
Resources	DELETE	/resources/{resource_id}	Delete Resource
Schedules	POST	/schedules	Add Schedule
Schedules	GET	/schedules/class/{class_id}	Get Class Schedule
Schedules	GET	/schedules/teacher/{teacher_id}	Get Teacher Schedule

Domain	Method	Endpoint	Purpose
Schedules	GET	/schedules/student/{student_id}	Get Student Schedule
Schedules	PUT	/schedules/{schedule_id}	Update Schedule
Schedules	DELETE	/schedules/{schedule_id}	Delete Schedule
Student Enrollments	POST	/enrollments	Create Enrollment
Student Enrollments	GET	/enrollments	Get All Enrollments
Student Enrollments	GET	/enrollments/student/{student_id}	Get Student Enrollments
Student Enrollments	GET	/enrollments/program/{program_id}	Get Program Enrollments
Student Enrollments	GET	/enrollments/class/{class_id}	Get Class Enrollments
Student Enrollments	PUT	/enrollments/{enrollment_id}	Update Enrollment
Student Enrollments	PATCH	/enrollments/{enrollment_id}/status	Change Enrollment Status
Student Enrollments	DELETE	/enrollments/{enrollment_id}	Delete Enrollment
Student Fees	POST	/student-fees	Create Fee
Student Fees	GET	/student-fees/overdue	Get Overdue Fees
Student Fees	GET	/student-fees	Get All Fees
Student Fees	GET	/student-fees/student/{student_id}	Get Student Fees
Student Fees	PUT	/student-fees/{fee_id}	Update Fee
Student Fees	DELETE	/student-fees/{fee_id}	Delete Fee
Students	POST	/students	Create Student
Students	GET	/students/search	Search Students
Students	GET	/students	Get All Students
Students	GET	/students/{student_id}	Get Student
Students	PUT	/students/{student_id}	Update Student
Students	PATCH	/students/{student_id}/status	Change Status
Students	DELETE	/students/{student_id}	Delete Student
Teacher Assignments	POST	/assignments	Create Assignment

Domain	Method	Endpoint	Purpose
Teacher Assignments	GET	/assignments	Get All Assignments
Teacher Assignments	GET	/assignments/class/{class_id}	Get Assignments By Class
Teacher Assignments	GET	/assignments/teacher/{teacher_id}	Get Assignments By Teacher
Teacher Assignments	PUT	/assignments/{assignment_id}	Update Assignment
Teacher Assignments	DELETE	/assignments/{assignment_id}	Delete Assignment
Teachers	POST	/teachers	Create Teacher
Teachers	GET	/teachers/search	Search Teachers
Teachers	GET	/teachers	Get All Teachers
Teachers	GET	/teachers/{teacher_id}	Get Teacher
Teachers	GET	/teachers/{teacher_id}/assignments	Get Teacher Assignments
Teachers	PUT	/teachers/{teacher_id}	Update Teacher
Teachers	DELETE	/teachers/{teacher_id}	Delete Teacher
User Transactions	POST	/transactions	Create Transaction
User Transactions	GET	/transactions	Get All Transactions
User Transactions	GET	/transactions/statement/{user_id}	Get User Statement
User Transactions	GET	/transactions/{transaction_id}	Get Transaction
User Transactions	PATCH	/transactions/{transaction_id}/status	Update Transaction Status
User Transactions	DELETE	/transactions/{transaction_id}	Delete Transaction
Users	POST	/users/login	Login
Users	POST	/users	Create User
Users	GET	/users	Get Users
Users	GET	/users/search	Search Users
Users	GET	/users/{user_id}	Get User
Users	PUT	/users/{user_id}	Update User
Users	PATCH	/users/{user_id}/status	Change Status

Domain	Method	Endpoint	Purpose
Users	DELETE	/users/{user_id}	Delete User

Security and Integrity

Security relies on several complementary mechanisms: authentication of active accounts, role-based dashboard separation, Pydantic input validation, parameterized SQL queries, and relational constraints in the database. Passwords are hashed before comparison during login.

Risk	Applied Measure
Unauthorized access	Redirection by role and separated interfaces by profile.
Invalid data	Pydantic schemas, frontend checks, and HTTP exceptions.
Relational inconsistencies	Foreign keys, dedicated managers, and domain-based tables.
Password exposure	The password field is removed from user API responses.
Operation errors	API failure messages, HTTP status codes, and frontend-side handling.

Error Management

Errors are handled at two levels. On the API side, routers raise HTTP exceptions when a resource does not exist, a request is invalid, or an operation fails. On the frontend side, services interpret responses and show readable feedback inside dashboards.

Repository and Deployment

The source code is organized in the following GitHub repository: https://github.com/MohammedBoure/SMS-FE-BE. The frontend can be served as a static application, while the FastAPI backend runs with Uvicorn. Port forwarding and remote-access configuration were prepared for demonstration.

Technical References

Reference	Use in the Project
FastAPI Documentation - https://fastapi.tiangolo.com/	Backend application, APIRouter structure, HTTP operations, CORS middleware, and WebSocket endpoint.
Pydantic Documentation - https://docs.pydantic.dev/	Request-body validation, schema modeling, and structured data transfer.
MySQL Documentation - https://dev.mysql.com/doc/	Relational database design, constraints, and SQL storage.
MDN Web Docs: JavaScript - https://developer.mozilla.org/en-US/docs/Web/JavaScript	Frontend JavaScript behavior, modules, fetch calls, and browser-side interactions.

Reference	Use in the Project
PlantUML Documentation - https://plantuml.com/	UML notation used for use case, class, sequence, and navigation diagrams.
ReportLab User Guide - https://docs.reportlab.com/reportlab/userguide/ch1_intro/	Programmatic generation of the academic PDF report.
Project Repository - https://github.com/MohammedBoure/SMS-FE-BE	Source-code reference for the implemented frontend, backend, database, and report-generation scripts.

Conclusion

The design combines a clear architecture, coherent relational models, UML diagrams, and separated application modules. The next chapter presents the implementation and validation.

Chapter 4 - Implementation and Testing

Introduction

This chapter presents the transition from design to implementation. It describes the development environment, source-code organization, main interfaces, and validation scenarios used to check the behavior of the system.

Development Environment

Element	Description
System	Development was carried out on Windows with a Python environment and a modern browser for frontend testing.
Backend	Python 3, FastAPI, Uvicorn, Pydantic, mysql-connector-python, python-dotenv, and python-multipart.
Frontend	HTML, CSS, vanilla JavaScript, JSON translation files, and static pages by role.
Database	MySQL with SQL scripts and Python data managers.
Documentation	PlantUML for diagrams, SVG export, and PDF generation with ReportLab.

Folder Architecture

Folder / File	Role
backend/app.py	FastAPI entry point and router inclusion.
backend/apis	REST endpoints by functional domain.
backend/database	Data managers, configuration, and MySQL access.
frontend/pages	HTML pages by role and login page.
frontend/js/core	Shared modules: API, authentication, routing, storage, preferences, and i18n.
frontend/js/roles	Controllers, services, and interfaces specific to each role.
frontend/locales	Translations by language and dashboard.
UML_projet	PlantUML sources, generation scripts, SVG assets, and PDF report sections.

Application Presentation

The following screenshots illustrate representative application screens: login, posts, grade management, messaging, educational resources, and parent follow-up. They complement the UML diagrams by showing the visible result for users.

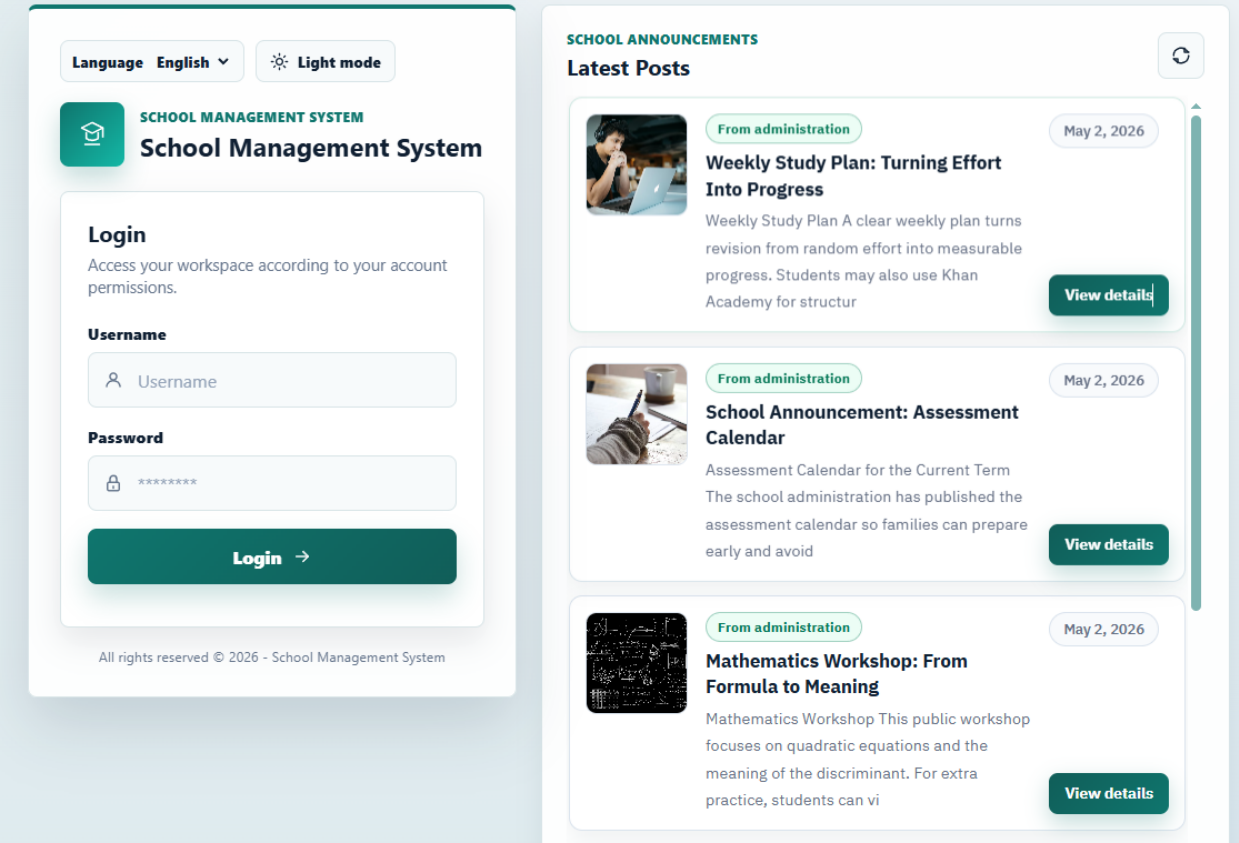

Figure 4.1 - Login Interface.

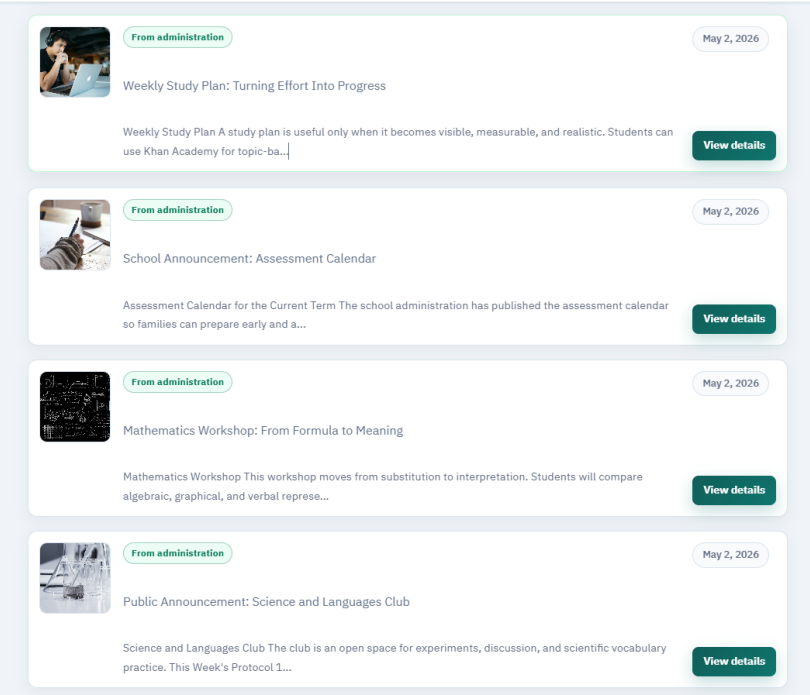

Figure 4.2 - Posts Board.

Students will work in teams to plant, observe, and document the growth of classroom plants. The project connects science, responsibility, and writing.

Role	Evidence to Submit
Observer	Height chart
Photographer	One image per week
Writer	80-word reflection

Plant growth can be modeled with a simple logistic curve:

$$h(t) = \frac{K}{1 + Ae^{-rt}}$$

Figure 4.3 - Post Details.

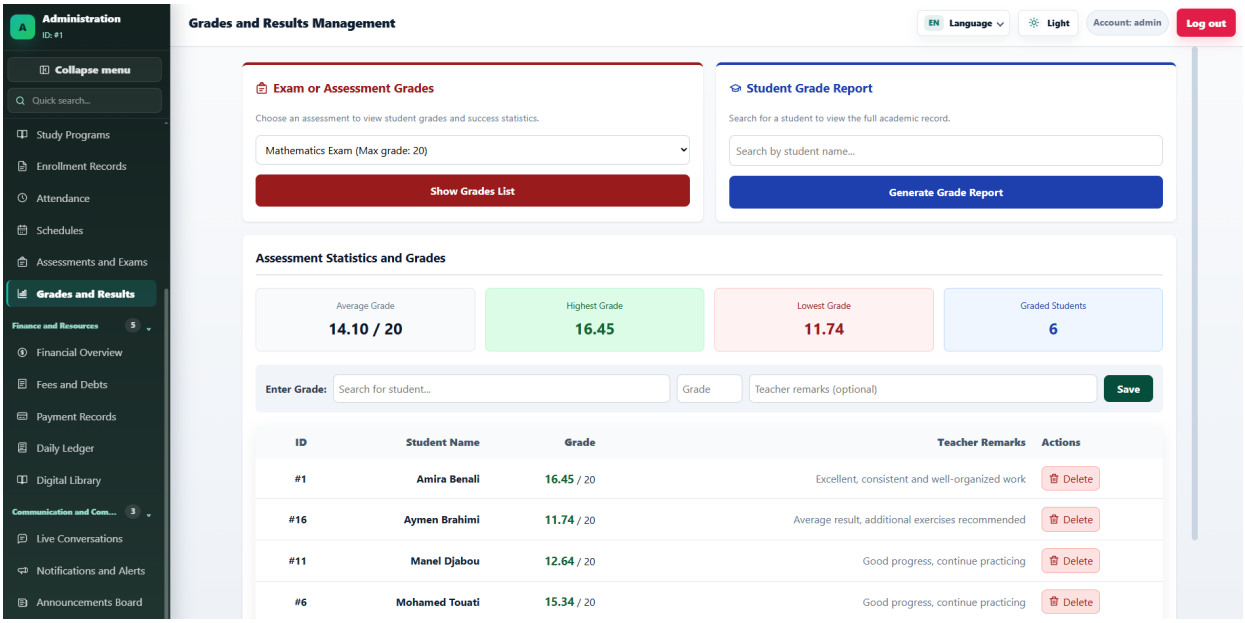

Figure 4.4 - Administrator Grade Management.

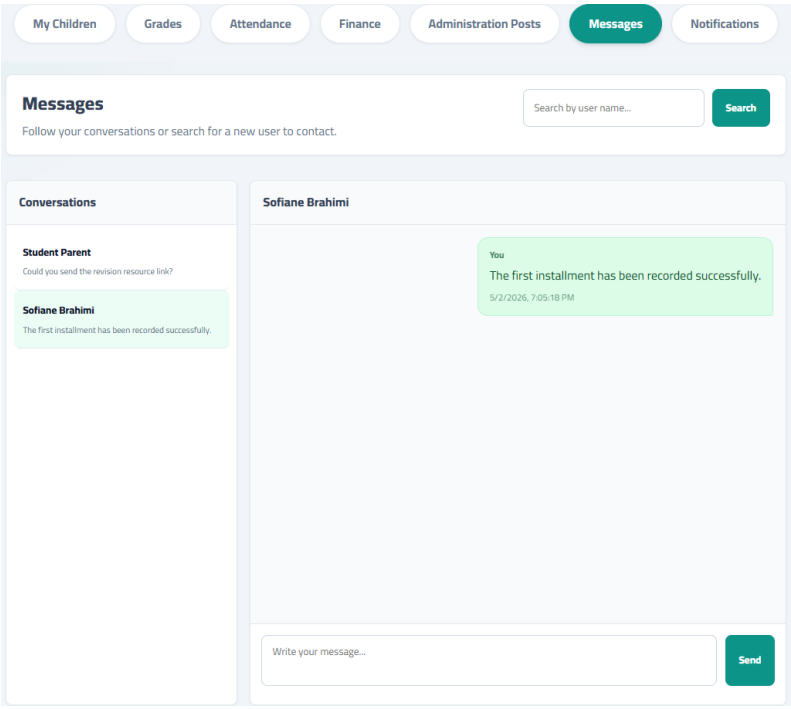

Figure 4.5 - Parent Messaging Interface.

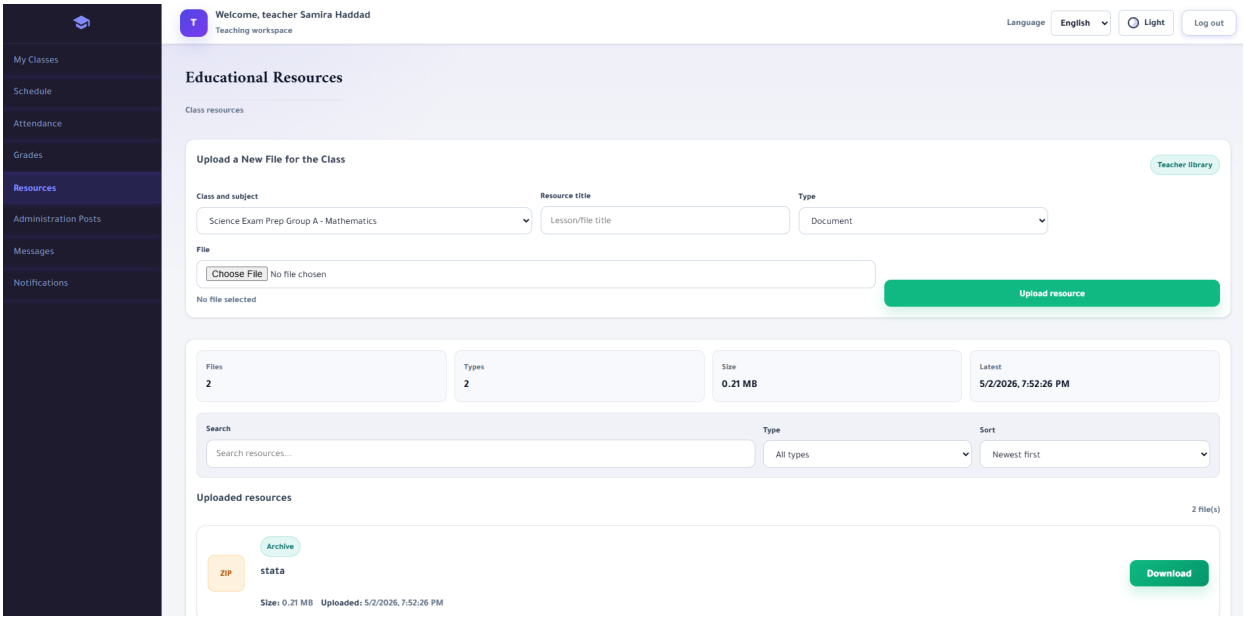

Figure 4.6 - Teacher Resources Interface.

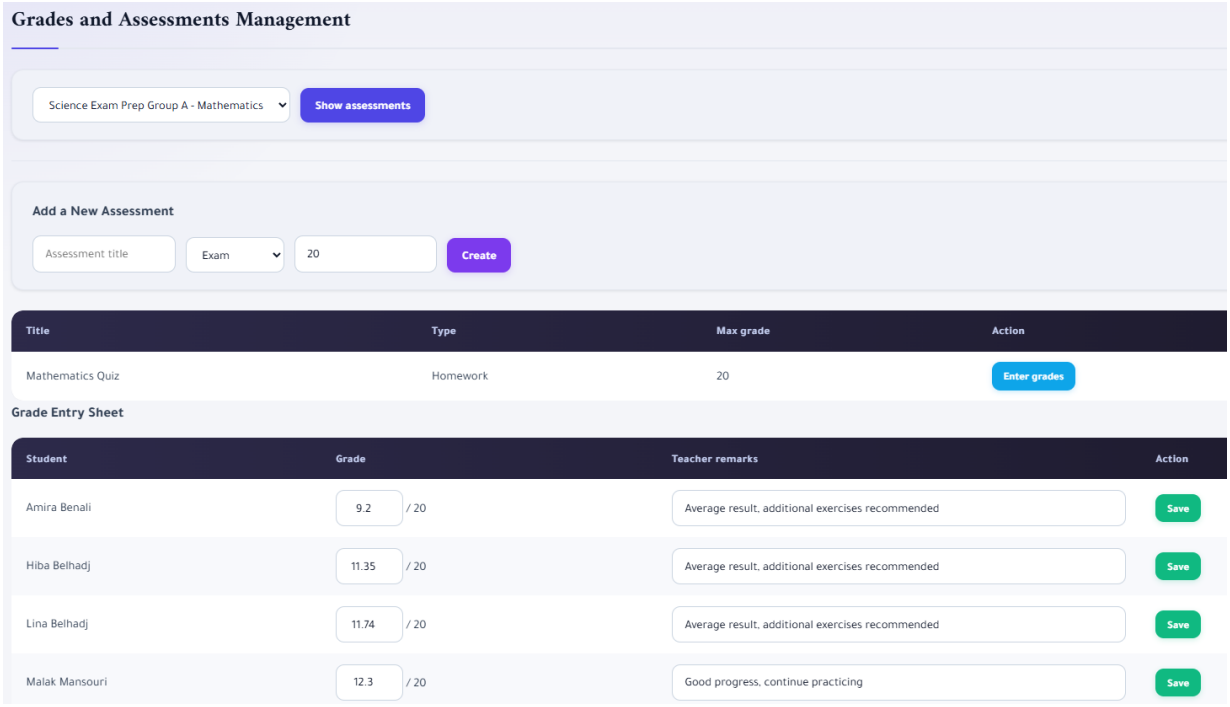

Figure 4.7 - Teacher Grade Entry and Consultation.

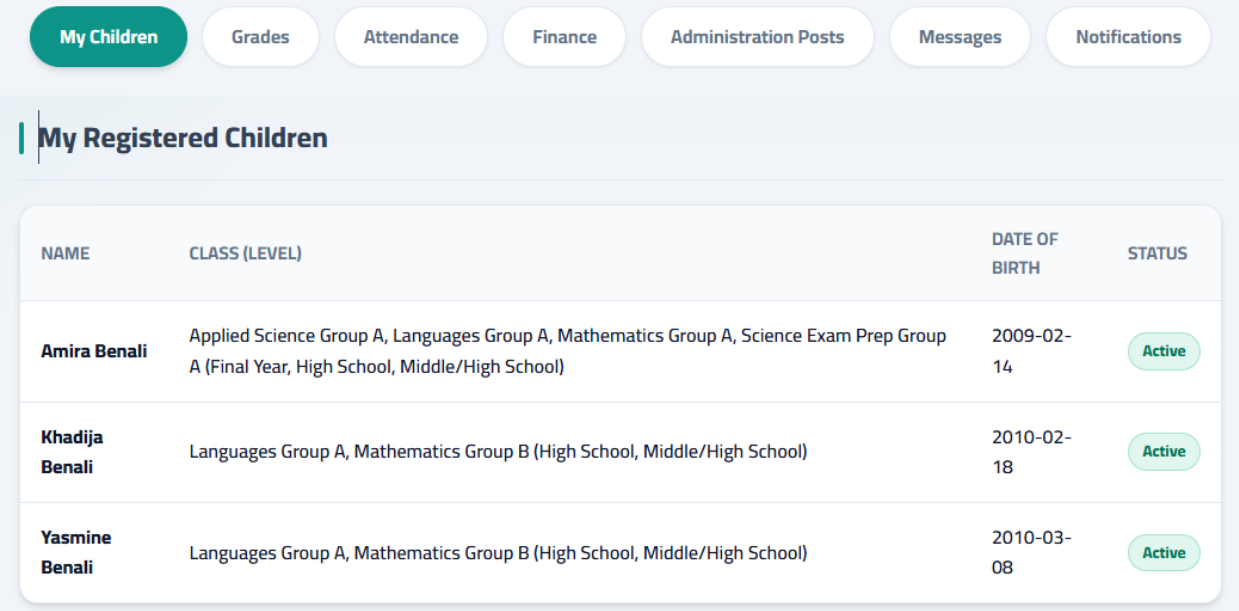

Figure 4.8 - Parent Children Follow-Up View.

Testing and Validation

Validation was organized around business scenarios covering the main roles. The tests confirm that data flows correctly between the interface, the API, and the database, and that functional rights remain coherent.

Scenario	Expected Result
Active user login	The corresponding dashboard opens and initial data is loaded.
Student file creation	The profile is saved, linked to parent/class data, and available in searches.
Attendance entry	The class sheet is updated and can be consulted by concerned profiles.
Grade entry	The grade is saved and displayed for teacher, student, parent, and administrator views.
Payment recording	The balance is updated and the payment history is visible in finance modules.
Message sending	The conversation is updated in the inboxes of the concerned users.
Language or theme change	The interface updates without losing the current user context.

Browser Validation

The screens were checked in a modern browser to verify navigation, forms, tables, modals, messaging, visual preferences, and the screenshots used in this report. APIs were verified through direct application calls and read/write scenarios.

Conclusion

The implementation confirms the feasibility of the designed architecture: roles have specialized dashboards, modules communicate with the REST API, and data is stored in a relational database. The validation scenarios show that the main business flows are covered.

General Conclusion

This work made it possible to design and implement a web-based school management application covering administrative, academic, financial, and communication dimensions. The project starts from a concrete need: replacing scattered management practices with a centralized, structured, and role-based platform.

The existing-system study revealed the limits of manual documents and disconnected tools. The requirements analysis identified actors, modules, and quality constraints. UML design formalized use cases, the structural model, interaction sequences, and navigation. Finally, implementation materialized these choices in a frontend/backend architecture connected to a MySQL database.

Project Assessment

Aspect	Result
Functional	Dedicated dashboards for administrator, receptionist, accountant, teacher, student, and parent.
Technical	RESTful architecture with JavaScript frontend, FastAPI backend, and MySQL relational database.
Modeling	Use case, class, sequence, and navigation diagrams integrated into the report.
User experience	Role-based interfaces, theme support, languages, messaging, notifications, and posts.
Documentation	Academic report structured around context, requirements, design, implementation, and testing.

Future Work

Perspective	Possible Improvement
Advanced security	Add token-based authentication, stronger password policies, and audit logging.
Reporting	Create PDF/Excel exports for grades, attendance, fees, and class statistics.
Deployment	Prepare a Docker environment and a reproducible production configuration.
Automated tests	Add backend unit tests, API integration tests, and frontend end-to-end tests.
Mobile experience	Improve responsive behavior or provide a dedicated mobile experience for parents and students.

In conclusion, the School Management System provides a solid basis for modernizing school administration. Its modular structure supports gradual evolution and keeps a clear connection between analysis, design, and implementation.